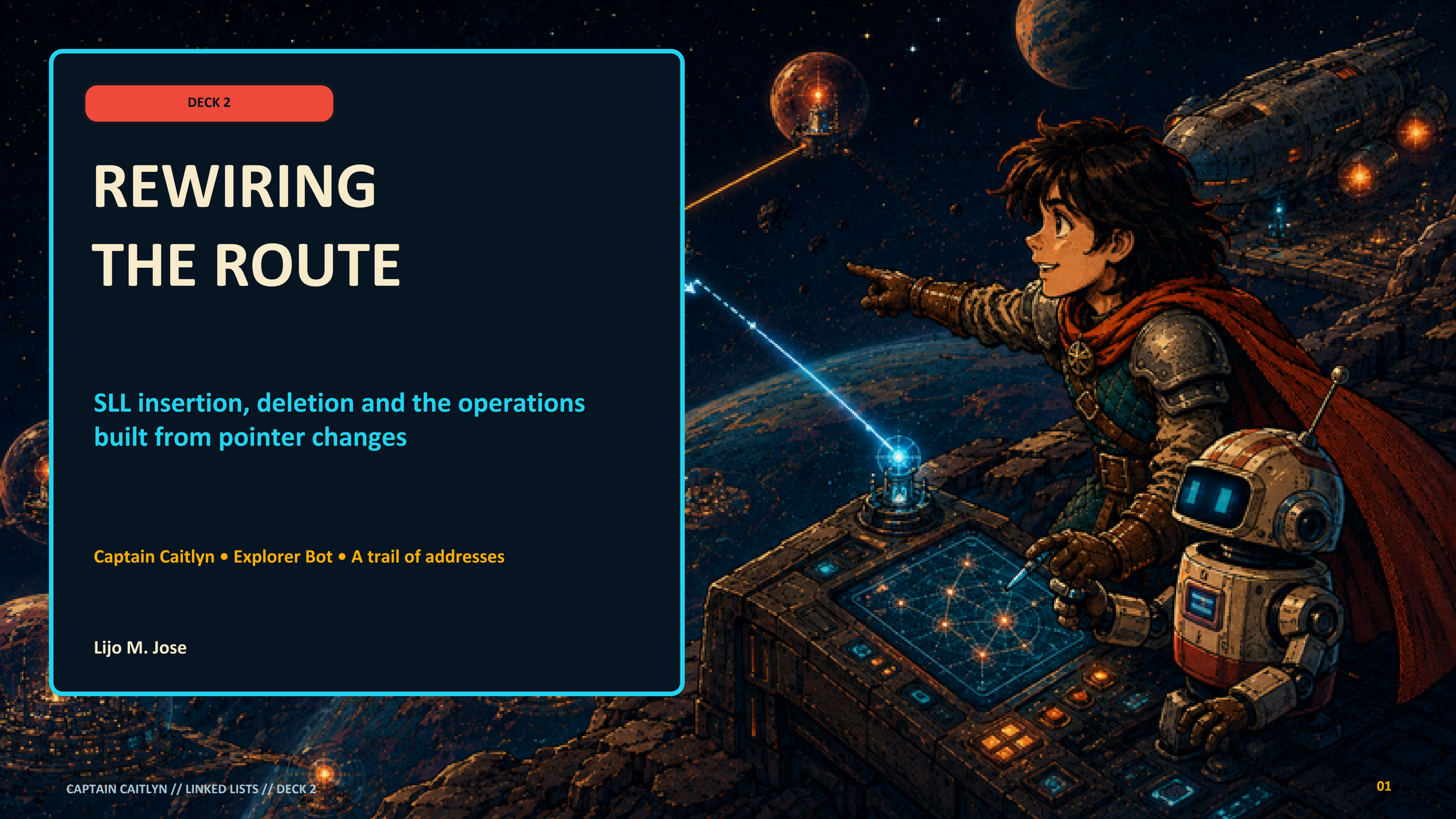
DECK 2

REWIRING THE ROUTE

SLL insertion, deletion and the operations built from pointer changes

Captain Caitlyn • Explorer Bot • A trail of addresses

Lijo M. Jose



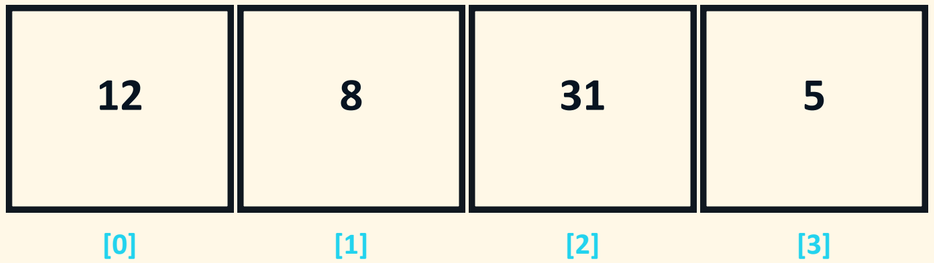
RECALL

A chain survives only while every needed address remains reachable

Today we change the chain without losing the part we still need.

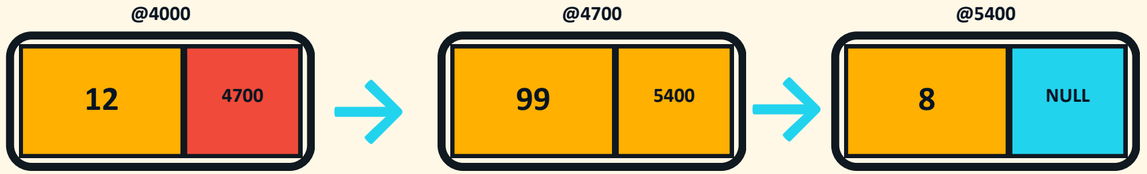
Arrays move values; linked lists redirect links

ARRAY INSERTION



shift the suffix

LINKED-LIST INSERTION

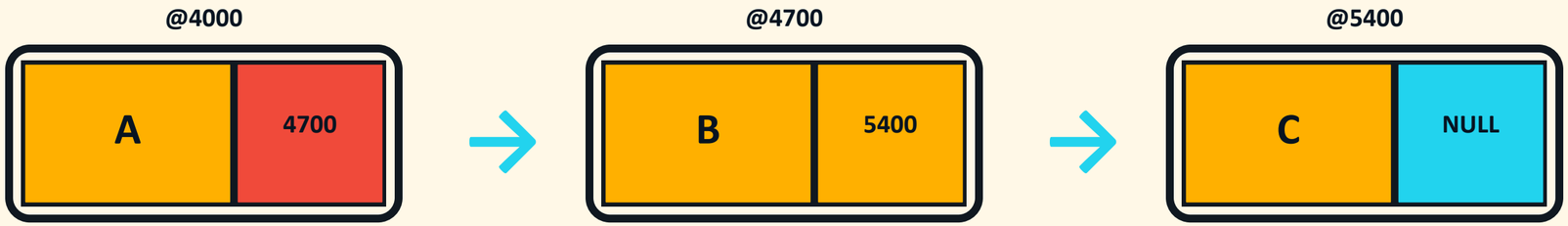


change a few addresses

Local rewiring is cheap only after the correct location is known.

SAFETY INVARIANT

The golden rule: save the route before overwriting it



If A->next is overwritten before 4700 is saved, B and C may become unreachable.

ASK BEFORE REVEALING

A new first beacon must point to the old first beacon

A `head = newNode; newNode->next = head;`

B `newNode->next = head; head = newNode;`

EXPLORER BOT

Which order preserves the old route?

PAUSE • TRACE BOTH

REVEAL

Insertion at the beginning needs two pointer writes

1 **newNode->next = head;**

Preserve the old entry address inside the new node.

2 **head = newNode;**

Move the entry pointer only after the new link is safe.

TIME: O(1)

The same code handles an empty list

INSERT FRONT

```
bool insertFront(Node **head, int value) {
    Node *newNode = createNode(value);
    if (newNode == NULL) return false;

    newNode->next = *head;
    *head = newNode;
    return true;
}
```

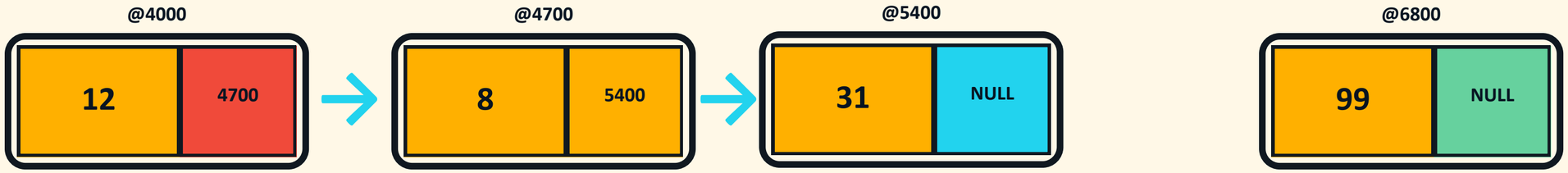
KEY IDEA

If *head is NULL, the new node simply stores NULL as its next.

Node **head is required because the function may change the caller's head.

INSERT AT END

Appending without a tail begins with a traversal



temp walks until temp->next == NULL

TRAVERSAL O(n)

QUICK CHECK

To append, stop on the last node—not after it

A while (temp != NULL)

B while (temp->next != NULL)

CAPTAIN CAITLYN

Which condition leaves temp pointing to the node whose next must change?

PAUSE • VOTE

Append is simple once the last node is found

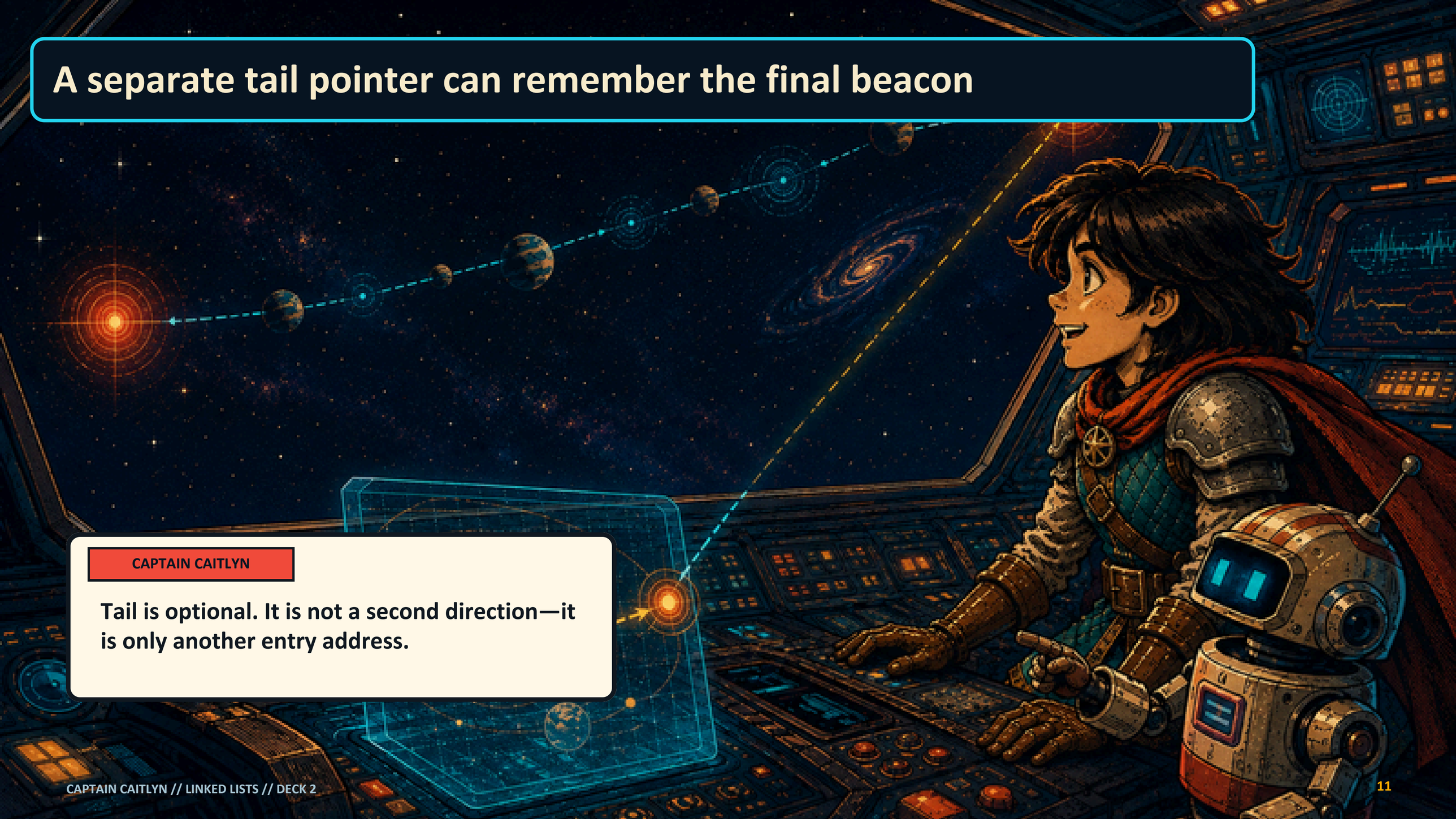
APPEND WITHOUT TAIL

```
bool append(Node **head, int value) {
    Node *newNode = createNode(value);
    if (newNode == NULL) return false;

    if (*head == NULL) {
        *head = newNode;
        return true;
    }
    Node *temp = *head;
    while (temp->next != NULL)
        temp = temp->next;
    temp->next = newNode;
    return true;
}
```

The new node was created with next = NULL.

O(n)

A separate tail pointer can remember the final beacon



CAPTAIN CAITLYN

Tail is optional. It is not a second direction—it is only another entry address.

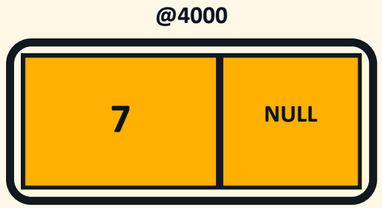
OPTIONAL OPTIMISATION

An SLL may keep both head and tail

EMPTY

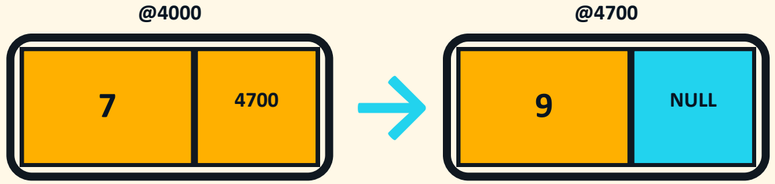
head = tail = NULL

ONE NODE



head == tail

MANY NODES



tail->next == NULL

Tail makes append O(1), but it must be repaired whenever the last node changes.

With tail, append becomes a constant-time update

APPEND WITH HEAD + TAIL

```
bool append(List *list, int value) {
    Node *newNode = createNode(value);
    if (newNode == NULL) return false;

    if (list->head == NULL)
        list->head = list->tail = newNode;
    else {
        list->tail->next = newNode;
        list->tail = newNode;
    }
    return true;
}
```

List stores two pointers:

```
Node *head;
Node *tail;
```

APPEND O(1)

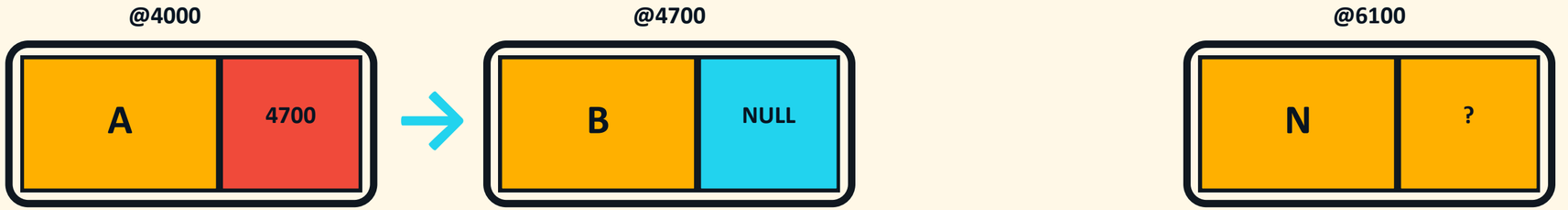
Insertion in the middle is a two-link handoff

EXPLORER BOT

The new beacon first learns the old successor.
Only then may the previous beacon point to it.

ASK THE CLASS

Which write must happen first?



A

A->next = N; N->next = B;

B

N->next = B; A->next = N;

Preserve the successor, then expose the new node

1 `newNode->next = current->next;`

The old successor is now safe.

2 `current->next = newNode;`

The previous node now enters the new route.

Reverse these writes and the old successor can be lost.

Insert-after turns search into a local rewiring operation

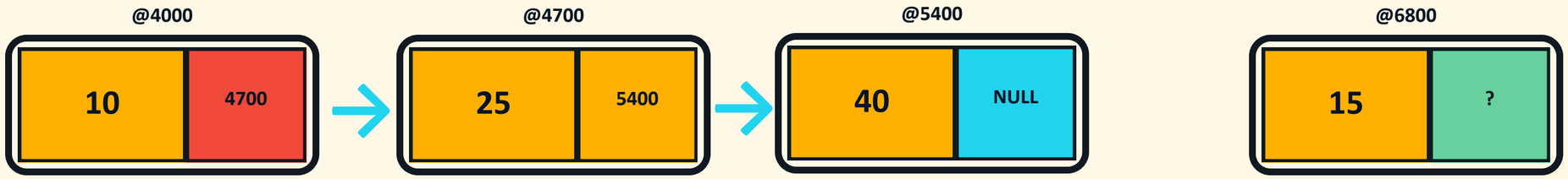
INSERT AFTER KEY

```
bool insertAfter(Node *head, int key, int value) {
    Node *current = head;
    while (current != NULL && current->data != key)
        current = current->next;

    if (current == NULL) return false;
    Node *newNode = createNode(value);
    if (newNode == NULL) return false;

    newNode->next = current->next;
    current->next = newNode;
    return true;
}
```

Search: O(n)

Rewire: O(1)

Insert-before needs the predecessor—or a special head case



To insert 15 before 25, we must change 10->next.

If key is at head, reuse insertFront.

Track prev and current together

INSERT BEFORE KEY

```
bool insertBefore(Node **head, int key, int value) {
    if (*head == NULL) return false;
    if ((*head)->data == key)
        return insertFront(head, value);


    Node *prev = *head;
    Node *current = prev->next;
    while (current != NULL && current->data != key) {
        prev = current;
        current = current->next;
    }
    if (current == NULL) return false;


    Node *newNode = createNode(value);
    if (newNode == NULL) return false;
    newNode->next = current;
    prev->next = newNode;
    return true;
}
```

prev follows one node behind current.

GENERALISE

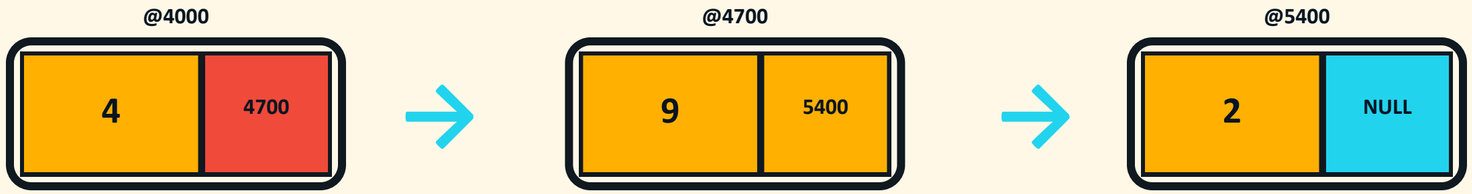
Insertion at a position is the same predecessor problem

position 1	position k > 1	invalid position
insertFront	walk to position k−1	stop if current == NULL

Once the predecessor is known, the actual insertion is still O(1).

Checkpoint: predict every changed pointer

Start:



- **Insert 7 at the beginning.**
- **Insert 6 after 9.**
- **Append 1.**

Draw the final links and state the time required for each operation—with and without tail.

Deletion is bypass first, free second

1 • IDENTIFY

node to remove + predecessor

2 • BYPASS

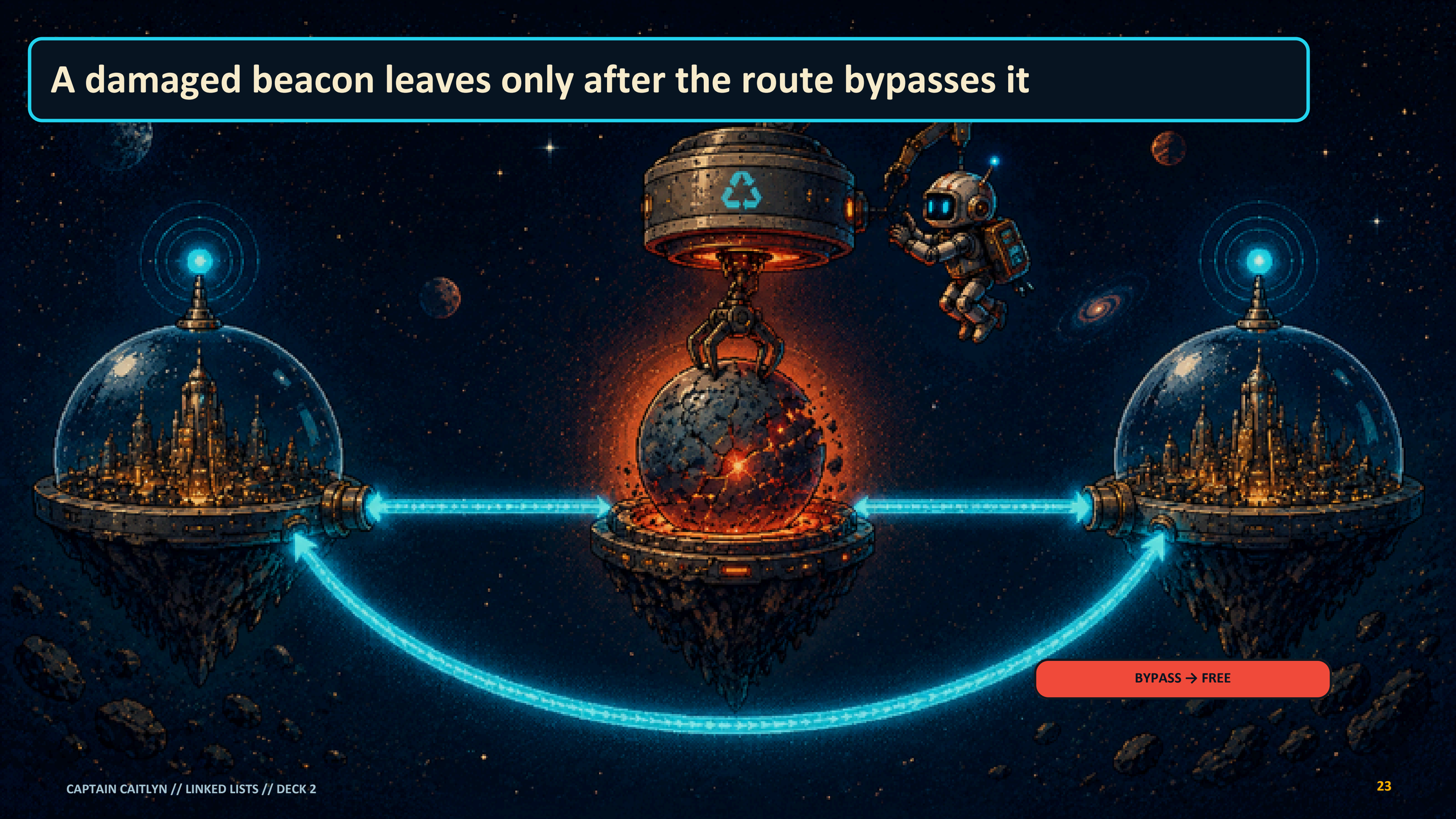
previous->next = target->next

3 • FREE

free(target)

Free too early and the successor address may disappear with the node.

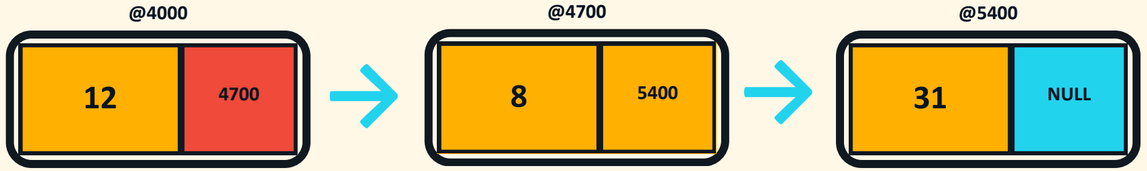
A damaged beacon leaves only after the route bypasses it



DELETE FRONT

Deleting the first node moves head forward

```
temp = head;

head = head->next;

free(temp);
```



head moves to 8

O(1)

Delete-front must also repair an optional tail

DELETE FRONT

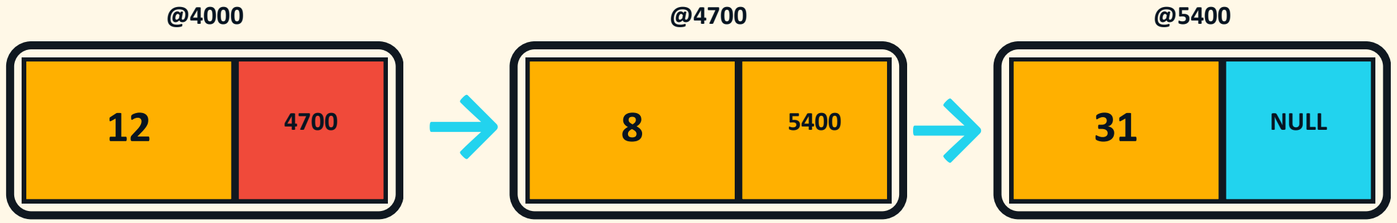
```
bool deleteFront(List *list) {
    if (list->head == NULL) return false;

    Node *target = list->head;
    list->head = target->next;
    free(target);

    if (list->head == NULL)
        list->tail = NULL;
    return true;
}
```

When the only node is removed:

head = tail = NULL

DELETE END

Deleting the last node requires its predecessor



prev

target

Even with tail, an SLL cannot move backward to find prev.

Delete-last has three structural cases

DELETE LAST

```
bool deleteLast(List *list) {
    if (list->head == NULL) return false;
    if (list->head == list->tail)
        return deleteFront(list);

    Node *prev = list->head;
    while (prev->next != list->tail)
        prev = prev->next;

    free(list->tail);
    list->tail = prev;
    list->tail->next = NULL;
    return true;
}
```

empty

one node

many nodes

STILL O(n)

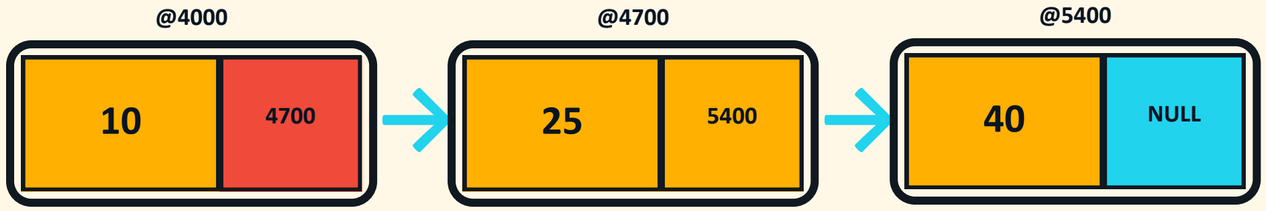
IMPORTANT DISTINCTION

Tail helps append, not predecessor discovery

Operation	head only	head + tail
append	O(n)	O(1)
read last	O(n)	O(1)
delete last	O(n)	O(n)

Why? tail has the last address, but no address of the node before it.

Deleting a specified value again needs prev and current



delete 25

prev = 10

target = 25

prev->next = target->next

The head case is the only deletion without a predecessor

DELETE FIRST MATCH

```
bool deleteKey(Node **head, int key) {
    if (*head == NULL) return false;
    if ((*head)->data == key) {
        Node *target = *head;
        *head = target->next;
        free(target);
        return true;
    }

    Node *prev = *head;
    Node *target = prev->next;
    while (target != NULL && target->data != key) {
        prev = target;
        target = target->next;
    }
    if (target == NULL) return false;
    prev->next = target->next;
    free(target);
    return true;
}
```

If a tail is stored and target == tail, update tail = prev.

BUG RADAR

Four deletion mistakes that quietly destroy correctness

FREE FIRST

successor address lost

NO HEAD CASE

prev does not exist

STALE TAIL

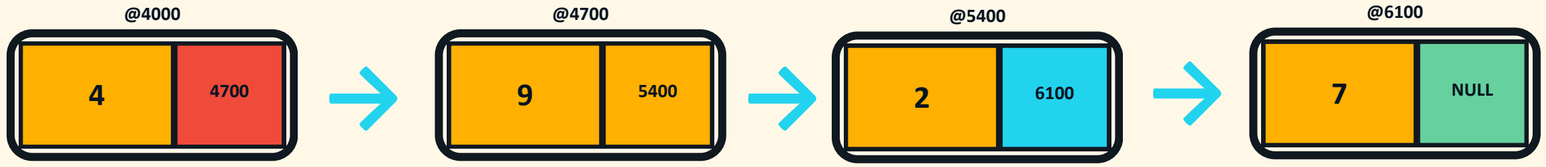
tail points to freed memory

USE AFTER FREE

target is no longer valid

Checkpoint: bypass, repair, then free

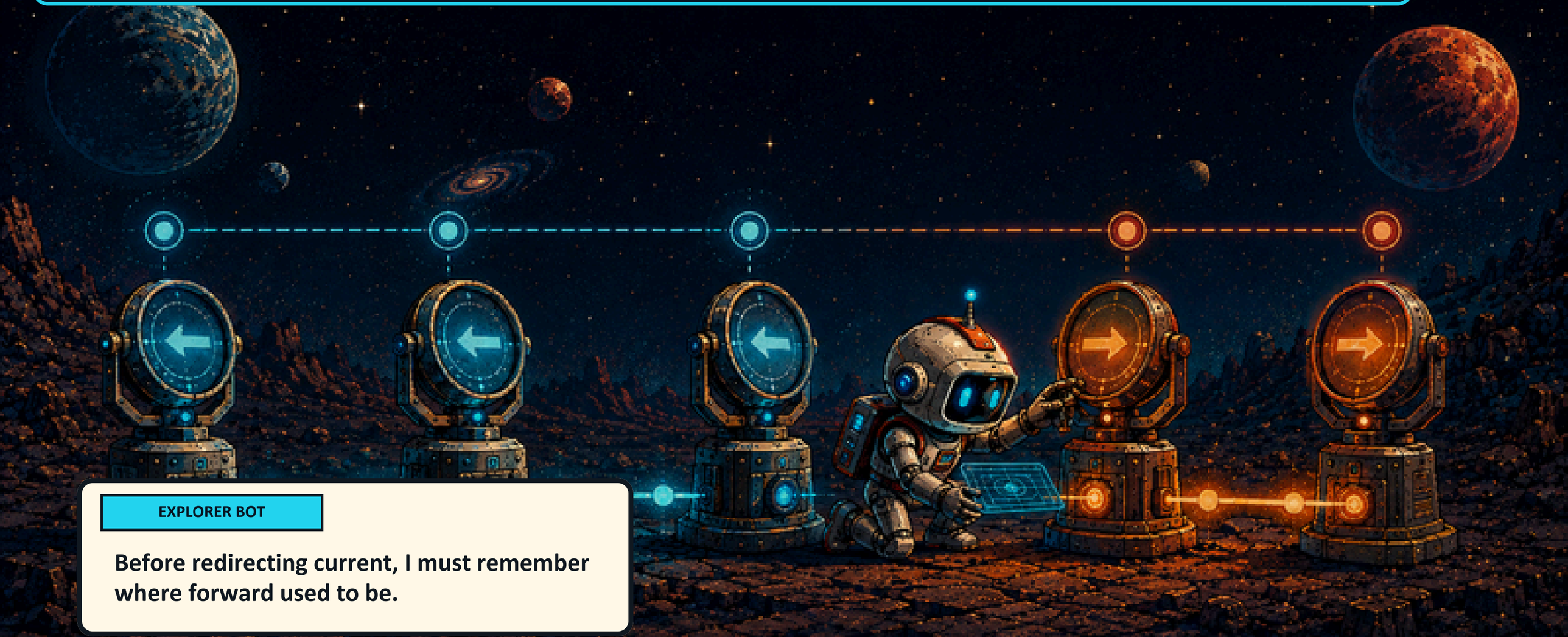
Start:



- Delete the first node.
- Delete the node containing 2.
- Delete the last remaining node.

For each case: name target, predecessor, changed link and freed address.

Reversal turns every forward link around—one at a time



EXPLORER BOT

Before redirecting current, I must remember where forward used to be.

Three pointers divide the route into past, present and future

prev

already reversed

current

node being changed

next

unprocessed route saved here

next = current->next

current->next = prev

prev = current; current = next

Trace one reversal step before trusting the loop

Before prev = NULL current = 4 next = ?

After one iteration prev = 4 current = 9 saved next = 9

and 4->next is now NULL

REVERSE CODE

Reverse is a disciplined four-line loop

REVERSE SLL

```
void reverse(Node **head) {
    Node *prev = NULL;
    Node *current = *head;

    while (current != NULL) {
        Node *next = current->next;
        current->next = prev;
        prev = current;
        current = next;
    }
    *head = prev;
}
```

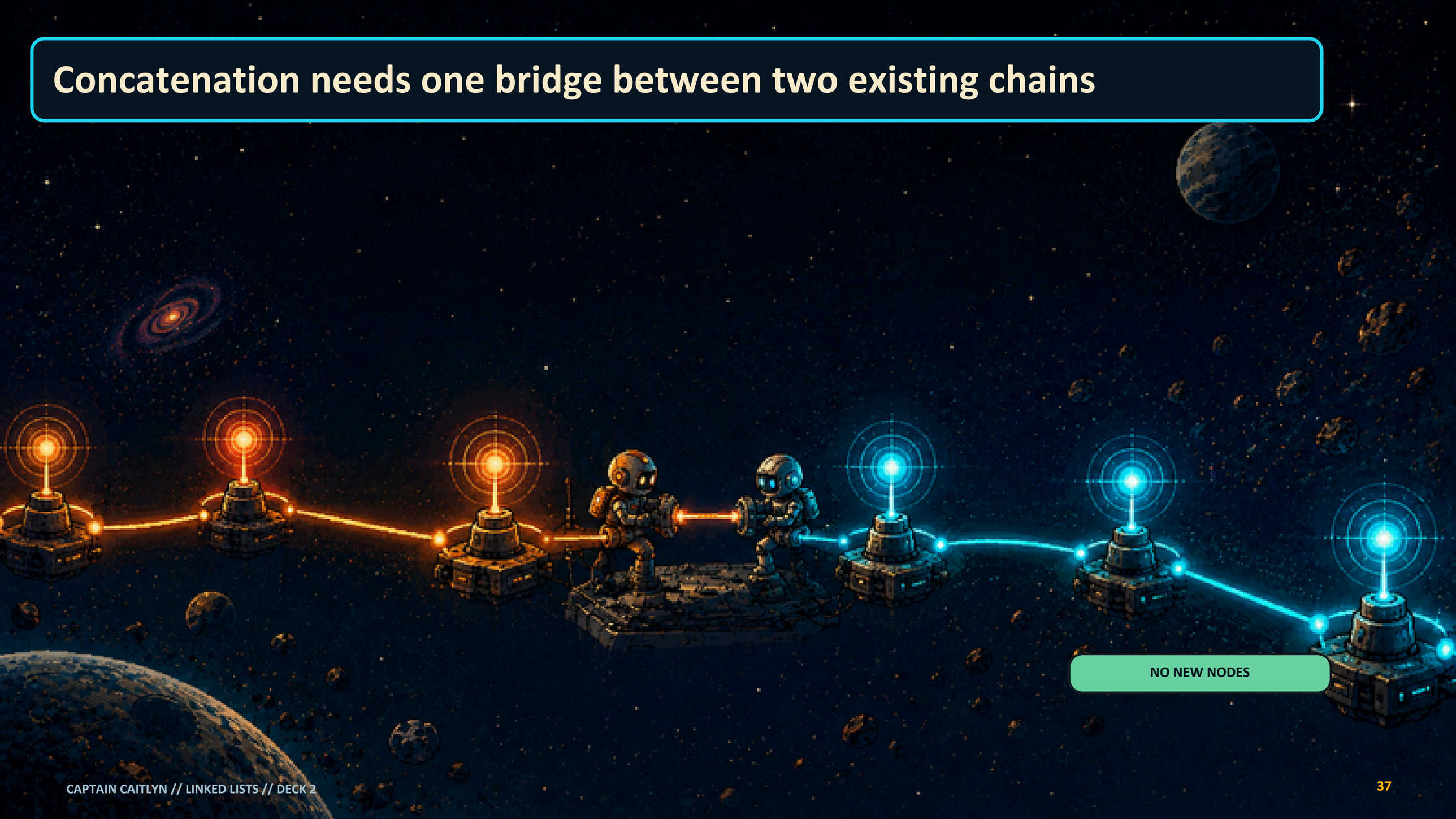
Time

O(n)

Extra space

O(1)

Concatenation needs one bridge between two existing chains



Connect the last node of list A to the head of list B

CONCATENATE

```
Node *concatenate(Node *headA, Node *headB) {
    if (headA == NULL) return headB;
    if (headB == NULL) return headA;

    Node *temp = headA;
    while (temp->next != NULL)
        temp = temp->next;
    temp->next = headB;
    return headA;
}
```

With tailA:

```
tailA->next = headB;

tailA = tailB;
```

O(1) WITH TAIL

Count traverses; erase consumes the route safely

COUNT

```
size_t count(const Node *head) {
    size_t n = 0;
    for (const Node *p = head; p != NULL; p = p->next)
        ++n;
    return n;
}
```

ERASE ALL

```
void erase(Node **head) {
    while (*head != NULL) {
        Node *target = *head;
        *head = target->next;
        free(target);
    }
}
```

Both are O(n). Erase leaves head == NULL.

Splitting by position distributes existing nodes

INPUT 10 → 20 → 30 → 40 → 50 → NULL

ODD POSITIONS 10 → 30 → 50 → NULL

EVEN POSITIONS 20 → 40 → NULL

Detach each node, then append it to the appropriate sublist.

Sorted insertion stops at the first larger value

12 → 25 → 40 → 60 → NULL

insert 28

Walk while current->data < 28

prev = 25 current = 40

Then perform the familiar insert-between rewiring.

SORTED MERGE

Merge two sorted lists by choosing the smaller front node

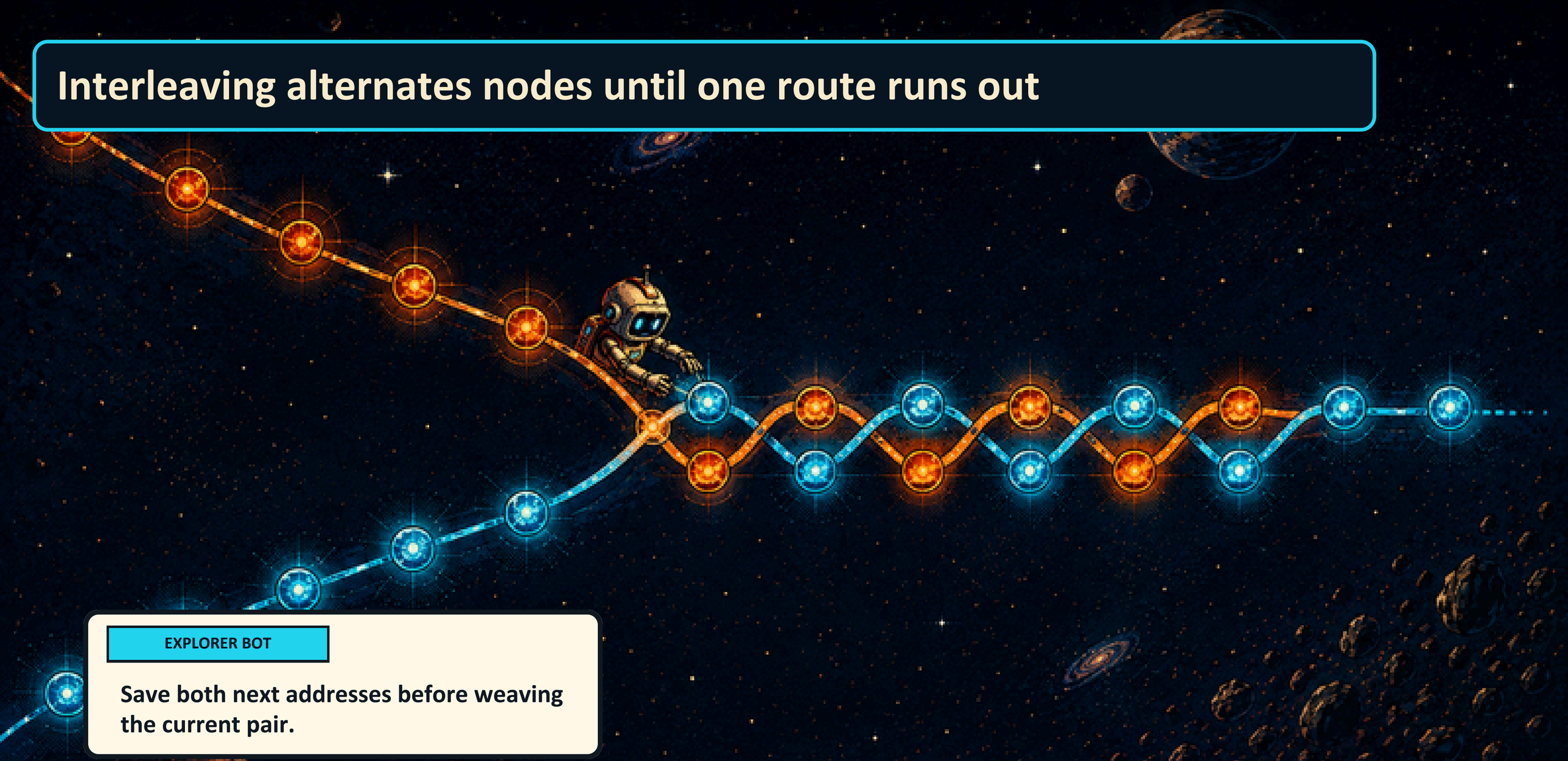
A 2 → 7 → 12 → NULL

B 3 → 5 → 15 → NULL

RESULT 2 → 3 → 5 → 7 → 12 → 15 → NULL

Relink existing nodes. Do not allocate copies; A and B cease to exist separately.

Interleaving alternates nodes until one route runs out



EXPLORER BOT

Save both next addresses before weaving the current pair.

Interleave is repeated insertion between consecutive nodes

ONE WEAVING STEP

```
nextA = a->next;
nextB = b->next;

a->next = b;
b->next = nextA;

a = nextA;
b = nextB;
```

If B still has nodes after A ends, attach the remainder after the last inserted B node.

Every node is visited once: O(n + m).

Delete-alternate and sorting are built from the same primitives

DELETE EVERY ALTERNATE NODE

CORE LOOP

```
Node *current = head;
while (current != NULL && current->next != NULL) {
    Node *target = current->next;
    current->next = target->next;
    free(target);
    current = current->next;
}
```

SORT A LINKED LIST

- Simpler: bubble-sort by swapping data.
- Stricter: relink nodes without moving payloads.
- For serious use: merge sort suits linked lists.

SIDE QUEST

Recursion can create or traverse the same chain

RECURSIVE TRAVERSAL

```
void displayRecursive(const Node *head) {
    if (head == NULL) {
        printf("NULL\n");
        return;
    }
    printf("%d -> ", head->data);
    displayRecursive(head->next);
}
```

Each call owns one node.

Base case: head == NULL

Recursive creation returns the head of the chain built by the remaining calls.

Cost: O(n) time and O(n) call stack

IMPLEMENTATION BLUEPRINT

A menu-driven program should delegate

main → read choice → call one operation

- One createNode function centralises allocation checks.
- Every delete function owns exactly one matching free.
- Display after each operation exposes pointer mistakes early.

LAB 4 READY

The search cost and the rewiring cost are separate

Operation	head only	head + tail
insert front	O(1)	O(1)
append	O(n)	O(1)
insert/delete by key	O(n)	O(n)
reverse / count / erase	O(n)	O(n)

Once a predecessor is known, insertion or deletion itself is O(1).

Coverage check: the complete SLL operation set

INSERT	front • end • after • before • position • sorted
DELETE	front • end • specified value • alternate • erase
TRANSFORM	reverse • concatenate • split • sort
COMBINE	sorted merge • interleave
TRAVERSE	display • count • recursive create/traverse

Every operation reduces to traversal, saved addresses, local rewiring and responsible free.

EXIT TICKET

Mission complete: safe rewiring preserves reachability

Explain without code:

- Why must insertion save the old successor first?
- Why must deletion bypass before free?
- Why does tail make append O(1) but not delete-last?

CAPTAIN CAITLYN

Next mission: Captain Caitlyn enters the route and needs a way to move both forward and back.